

CSE 309: Theory of Automata

Computability Theory

Shahid Hussain

shahidhussain@iba.edu.pk

March 2024

School of Mathematics and Computer Science
Institute of Business Administration

Turing Machines

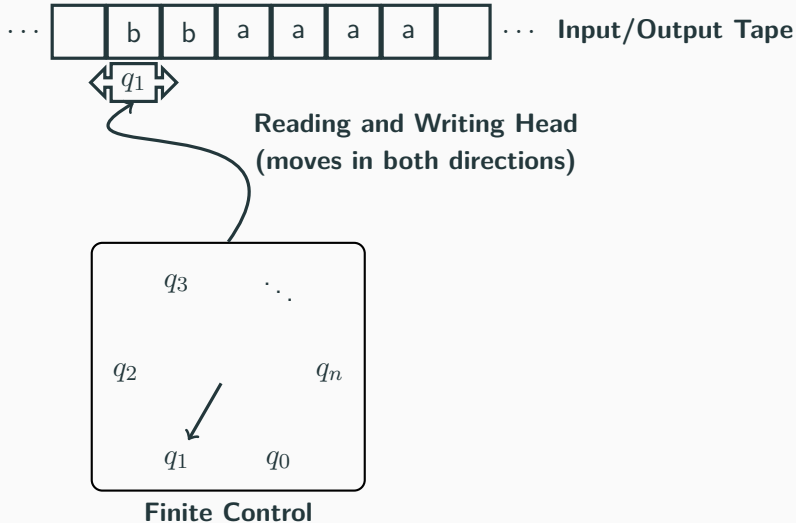
Turing Machines

- Turing machines are much more powerful models
- Similar to finite automata but with an unlimited and unrestricted memory
- A Turing machine is a much more accurate model of a general purpose computer

Turing Machines

- Turing machine model uses an infinite tape as its unlimited memory
- It has a tape head that can read and write symbols and move around on the tape
- Initially, the tape contains only the input string and blank everywhere
- Machine can continue computing until it decides to produce an output
- There are designated **accept** and **reject** states
- If TM does not enter an accepting or rejecting state it will go on forever, never halting

Turing Machines



Turing Machines vs Finite Automata

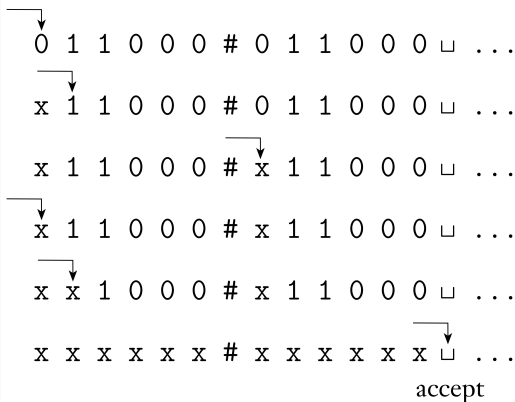
1. A Turing machine can both write on the tape and read from it
2. The read-write head can move both on the left and to the right
3. The tape is infinite
4. The special states for rejecting and accepting take effect immediately

Example Turing Machine

- Let $L = \{w\#w : w \in \{0,1\}^*\}$ and M is a TM that tests membership for L
- That is, we want M to accept w if $w \in L$ and reject otherwise

Example Turing Machine

- Let $L = \{w\#w : w \in \{0,1\}^*\}$ and M is a TM that tests membership for L
- That is, we want M to accept w if $w \in L$ and reject otherwise
- Assume $x = 01100\#01100$



Turing Machines

Turing Machine

A **Turing Machine** is a 7-tuple, $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where

1. Q is the set of states,
2. Σ is the input alphabet not containing the **blank symbol** $\sqcup$,
3. Γ is the tape alphabet where $\sqcup \in \Gamma$ and $\Sigma \subseteq \Gamma$,
4. $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function,
5. q_0 is the state state,
6. $q_{\text{accept}} \in Q$ is the accept state, and
7. $q_{\text{reject}} \in Q$ is the reject state, where $q_{\text{reject}} \neq q_{\text{accept}}$.

Computation by a Turing Machine

- Let $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ be a Turing Machine
- M receives the input $w = w_1w_2 \dots w_n \in \Sigma^*$ on the leftmost n squares of the tape (rest of the tape is blank)
- Once M starts the computation proceeds according to the transition function
- Computation continues until it enters either the accept or reject states

Computation by a Turing Machine

- At any instance, the current state, the current tape contents, and the head location together are called a **configuration** of TM
- Example configuration: $1011q_701111 = u q v$, current state is q i.e., q_7 , current tape content is uv i.e., 101101111 , and the head is currently on the second 0.
- Suppose configuration C_1 **yields** configuration C_2 if the TM can legally go from C_1 to C_2 in a single step
- That is, suppose we have $a, b, c \in \Gamma$ and $u, v \in \Gamma^*$ and states q_i and q_j , $u a q_i b v$ yields $u q_j a c v$ if

$$\delta(q_i, b) = (q_j, c, D)$$

where $D = L$ if M moves leftward and $D = R$ if M moves rightward

Turing Recognizable

A language L is called **Turing-recognizable** if some Turing machine recognizes it. L is also called **recursively enumerable language**.

Turing Machine

Turing Recognizable

A language L is called **Turing-recognizable** if some Turing machine recognizes it. L is also called **recursively enumerable language**.

Turing Decidable

A language L is called **Turing-decidable** or **decidable** if some Turing machine decides it. L is also called **recursive language**.

Turing Recognizable

A language L is called **Turing-recognizable** if some Turing machine recognizes it. L is also called **recursively enumerable language**.

Turing Decidable

A language L is called **Turing-decidable** or **decidable** if some Turing machine decides it. L is also called **recursive language**.

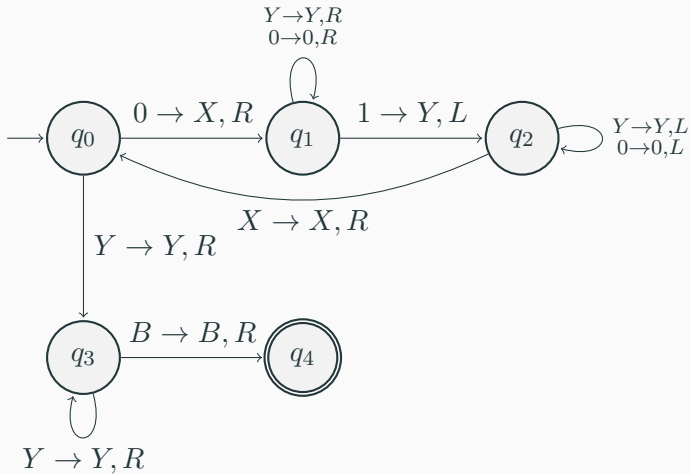
- Every decidable language is Turing-recognizable.
- We will see examples of languages that are Turing-recognizable but not decidable later on.

Turing Machines

- Let $L = \{0^n 1^n : n \geq 1\}$

state	Symbols				
	0	1	X	Y	B
q_0	(q_1, X, R)	—	—	(q_3, Y, R)	—
q_1	$(q_1, 0, R)$	(q_2, Y, L)	—	(q_1, Y, R)	—
q_2	$(q_2, 0, L)$	—	(q_0, X, R)	(q_2, Y, L)	—
q_3	—	—	—	(q_3, Y, R)	(q_4, B, R)
q_4	—	—	—	—	—

Turing Machine



Turing Machine

- Let $L = \{a^i b^j c^k : i \times j = k \wedge i, j, k \geq 1\}$
- TM M on input string w :
 1. Scan the input from left to right to determine whether it is a member of $a^* b^* c^*$ and **reject** otherwise
 2. Return the head to the left-hand end of the tape
 3. Cross off an a and scan to the right until a b occurs. Shuttle between the b 's and c 's crossing off one of each until all b 's are gone. If all c 's have been crossed off and some b 's remain **reject**
 4. Restore the crossed off b 's and repeat stage 3 if there is another a to cross off. If all a 's have been crossed off, determine whether all c 's also have been crossed off. If yes **accept**, **reject** otherwise

Multitape Turing Machines

- $\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k$
- where k is the number of tapes, the expression

$$\delta(q_i, a_1, \dots, a_k) = (q_j, b_1, \dots, b_k, L, R, \dots, L)$$

means if TM is in state q_i and heads 1 through k reading symbols a_1 through a_k , the machine goes to state q_j , writes symbols b_1 through b_k and directs each head to move left, right, or stay put as specified

Theorem 1

Every multitape Turing machine has an equivalent single-tape Turing machine.

Theorem 1

Every multitape Turing machine has an equivalent single-tape Turing machine.

Corollary 2

A language is Turing-recognizable if and only if some multitape Turing machine recognizes it.

Nondeterministic Turing Machines

- An NTM is defined in the expected way
- At any point in computation the machine may proceed according to several possibilities
- Transition function for NTM:

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \{L, R\})$$

Nondeterministic Turing Machines

- An NTM is defined in the expected way
- At any point in computation the machine may proceed according to several possibilities
- Transition function for NTM:

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma\{L, R\})$$

Theorem 3

Every nondeterministic Turing machine has an equivalent deterministic Turing machine.

Equivalence and Algorithms

Equivalence with Other Models

- Many different variants of Turing machines are equivalent in computational power
- In fact other computational models which are very different that TM's are also equivalent as long as they satisfy some reasonable requirements such as the ability to perform only a finite amount of work in a single step
- These models are often called *conventional models of computation*
- This has a philosophical implication
- Due to equivalence of computational models the class of algorithms that these models describe remains the same

Hilbert's 10-th Problem

- In 1900, David Hilbert in his famous lecture identified twenty five mathematical problems as a challenge for the 20-th century mathematicians
- On the problems in this 25 problem list, specifically the 10-th problem (famously known as Hilbert's tenth problem) as an important question, the problem describes “devise an algorithm that tests whether a polynomial has an integral root.”
- Hilbert did not use the word algorithm but he assumed there must exist such a procedure

Hilbert's 10-th Problem

- In 1900, David Hilbert in his famous lecture identified twenty five mathematical problems as a challenge for the 20-th century mathematicians
- On the problems in this 25 problem list, specifically the 10-th problem (famously known as Hilbert's tenth problem) as an important question, the problem describes “devise an algorithm that tests whether a polynomial has an integral root.”
- Hilbert did not use the word algorithm but he assumed there must exist such a procedure
- Unfortunately, no such algorithm/procedure exists
- This cannot be understood unless the formal definition of algorithms is known which Alonzo Church and Alan Turing defined
- Which later became **Church-Turing** thesis

Hilbert' 10-th Problem

- We can define this problem with differnt language
- Let $D = \{p : p \text{ is a polynomial with an itegral root}\}$
- The question then becomes, **Is D decidable?**
- Of course the answer is **no**
- However, D is Turing-recognizable
- We can construct such a TM, **HOW?**